

BABU BANARASI DAS UNIVERSITY

SCHOOL OF COMPUTER APPLICATIONS

NoSQL PROJECT REPORT

STUDENT MANAGEMENT SYSTEM USING MONGODB

Submitted By	Diwakar Mishra
Roll No.	33 Enrollment No. 1250258198
Class	BCA DS & AI 23
Submitted To	Mr. Harendra Singh Rajpoot
Academic Session	2026–2027
Submission Date	25 September 2026

A complete practical report with MongoDB query screenshots, dataset, operations, outputs and explanations.

CERTIFICATE

This is to certify that **Diwakar Mishra**, Roll No. **33**, Enrollment No. **1250258198**, Class **BCA DS & AI 23**, has completed the NoSQL project titled “**Student Management System Using MongoDB**” under the guidance of **Mr. Harendra Singh Rajpoot** during the academic session 2026–2027.

Student Signature

Teacher Signature

ACKNOWLEDGEMENT

I sincerely express my gratitude to my teacher **Mr. Harendra Singh Rajpoot** for giving me the opportunity to work on this NoSQL project titled "Student Management System Using MongoDB".

This project helped me understand the basic concepts of NoSQL databases and learn how MongoDB is used to create, retrieve, update, sort and delete data. I also learned how comparison operators such as greater than, less than and equal to, along with logical operators such as AND and OR, are used in MongoDB.

I am thankful to my teacher for the guidance and support provided throughout the completion of this project.

TABLE OF CONTENTS

1	Introduction and Objectives
2	Tools / Requirements
3	Methodology
4	Database and Collection Creation
5	Student Dataset
6	MongoDB Operations and Screenshots
7	Operators Used
8	Conclusion
9	References

1. INTRODUCTION

MongoDB is a popular NoSQL database management system that stores data in the form of documents. Unlike traditional relational databases, MongoDB does not require data to be stored in fixed rows and columns.

In this project, a Student Management System has been created using MongoDB. The database contains information about 50 students, including roll number, name, age, marks and city. Different MongoDB operations have been demonstrated, including inserting, displaying, updating, deleting, sorting, limiting, counting and searching records.

OBJECTIVES

- Understand the concept of NoSQL databases.
- Learn the basic features of MongoDB.
- Create a database and collection.
- Insert 50 student records.
- Perform CRUD operations.
- Use comparison operators.
- Perform sorting and filtering.
- Understand AND and OR operations.

2. TOOLS / REQUIREMENTS

- MongoDB / MongoDB Shell or MongoDB Compass
- A computer system
- Basic knowledge of database commands
- Student dataset used in this project

3. METHODOLOGY

The project follows a simple database workflow: create or switch to the Students database → create the students collection → insert student documents → retrieve records → apply comparison and logical filters → update and delete records → sort, limit, search and count documents → document the results.

4. STUDENT DATASET

The reference project contains 50 student documents with five fields: roll, name, age, marks and city.

Roll	Name	Age	Marks	City
1	Anamika	19	88	Lucknow
2	Uday	18	76	Jaunpur
3	Vaishnavi	19	92	Kanpur
4	Annu	18	69	Ayodhya
5	Vansh	19	81	Varanasi
6	Amrita	20	85	Lucknow
7	Naveen	19	73	Prayagraj
8	Pratik	19	90	Lucknow
9	Sangita	20	78	Gorakhpur
10	Vivek	18	65	Kanpur
11	Tarkeshwar	20	71	Jaunpur
12	Shreya	19	95	Lucknow
13	Akshita	18	84	Ayodhya
14	Shruti	19	87	Prayagraj
15	Kartik	20	74	Varanasi
16	Sejal	18	89	Lucknow
17	Astha	19	82	Kanpur
18	Jai	18	58	Gonda
19	Harshita	20	91	Lucknow
20	Omkar	19	63	Bareilly
21	Ayush	18	77	Lucknow
22	Saumya	19	86	Jaunpur
23	Vidya	20	72	Kanpur
24	Adarsh	19	80	Ayodhya
25	Anmol	18	67	Prayagraj

Roll	Name	Age	Marks	City
26	Anushka	20	94	Lucknow
27	Ananya	19	88	Varanasi
28	Bhaskar	18	61	Gorakhpur
29	Rohan	19	75	Kanpur
30	Nishant	20	79	Lucknow
31	Vineet	18	68	Jaunpur
32	Anika	19	90	Lucknow
33	Jai	20	55	Ayodhya
34	Lakshya	18	83	Kanpur
35	Krishna	19	96	Prayagraj
36	Rumana	20	70	Lucknow
37	Priyanshu	18	81	Varanasi
38	Aditya	19	66	Gorakhpur
39	Dhruv	20	93	Lucknow
40	Nivedita	19	85	Kanpur
41	Meenu	18	74	Jaunpur
42	Akanksha	20	89	Lucknow
43	Sarika	19	62	Ayodhya
44	Anchal	18	78	Prayagraj
45	Umra	20	84	Lucknow
46	Nashra	19	73	Kanpur
47	Aman	18	59	Gonda
48	Priya	20	87	Lucknow
49	Rahul	19	64	Bareilly
50	Neha	18	91	Varanasi

5. CREATE DATABASE

MongoDB query shown as a terminal-style screenshot:

```

MongoDB Shell | Create database

> use Students

switched to db Students

```

Explanation: Creates or switches to the Students database.

6. CREATE COLLECTION

MongoDB query shown as a terminal-style screenshot:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The title bar reads "MongoDB Shell | Create collection". The prompt is ">". The command entered is "db.createCollection('students')". The output is "{ ok: 1 }".

```
MongoDB Shell | Create collection

> db.createCollection("students")

{ ok: 1 }
```

Explanation: Creates a collection named students.

7. INSERT 50 STUDENTS

MongoDB query shown as a terminal-style screenshot:

A terminal window with a dark background and light-colored text. The title bar at the top shows three colored circles (red, yellow, green) on the left. The text inside the terminal reads: 'MongoDB Shell | Insert 50 students' on the first line, followed by a prompt '>' and the command 'db.students.insertMany([... 50 student documents ...])'. The next line shows the output: '{ acknowledged: true, insertedIds: { "0": ObjectId("..."), "1": ObjectId("..."), ... } }'. The final line shows the result: 'Result: 50 student documents inserted'.

```
MongoDB Shell | Insert 50 students

> db.students.insertMany([ ... 50 student documents ... ])

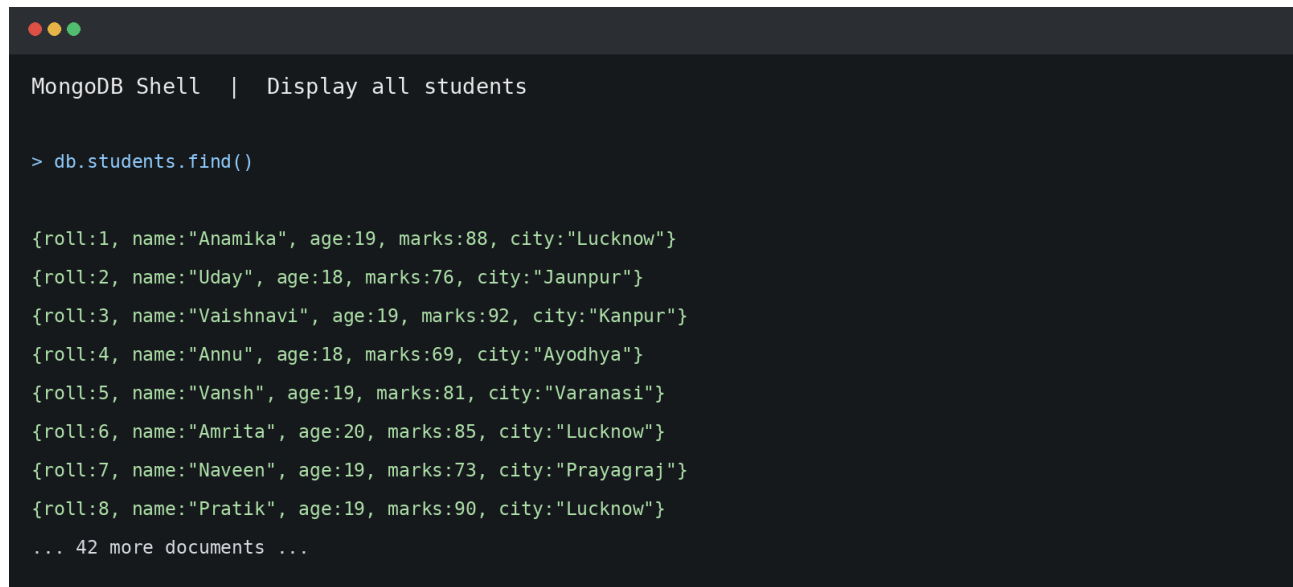
{ acknowledged: true,
  insertedIds: { "0": ObjectId("..."), "1": ObjectId("..."), ... } }

Result: 50 student documents inserted
```

Explanation: Inserts the 50 student documents into the collection.

8. DISPLAY ALL STUDENTS

MongoDB query shown as a terminal-style screenshot:



```
MongoDB Shell | Display all students

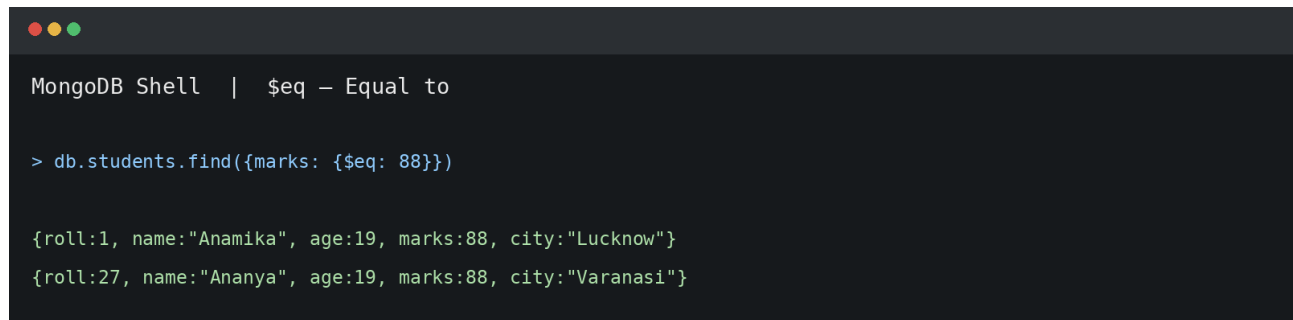
> db.students.find()

{roll:1, name:"Anamika", age:19, marks:88, city:"Lucknow"}
{roll:2, name:"Uday", age:18, marks:76, city:"Jaunpur"}
{roll:3, name:"Vaishnavi", age:19, marks:92, city:"Kanpur"}
{roll:4, name:"Annu", age:18, marks:69, city:"Ayodhya"}
{roll:5, name:"Vansh", age:19, marks:81, city:"Varanasi"}
{roll:6, name:"Amrita", age:20, marks:85, city:"Lucknow"}
{roll:7, name:"Naveen", age:19, marks:73, city:"Prayagraj"}
{roll:8, name:"Pratik", age:19, marks:90, city:"Lucknow"}
... 42 more documents ...
```

Explanation: Retrieves documents from the students collection.

9. EQUAL TO — \$eq

MongoDB query shown as a terminal-style screenshot:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The text inside the terminal is as follows:

```
MongoDB Shell | $eq – Equal to

> db.students.find({marks: {$eq: 88}})

{roll:1, name:"Anamika", age:19, marks:88, city:"Lucknow"}
{roll:27, name:"Ananya", age:19, marks:88, city:"Varanasi"}
```

Explanation: Finds students whose marks are exactly 88.

10. GREATER THAN — \$gt

MongoDB query shown as a terminal-style screenshot:

```
MongoDB Shell | $gt – Greater than

> db.students.find({marks: {$gt: 85}})

{roll:1, name:"Anamika", age:19, marks:88, city:"Lucknow"}
{roll:3, name:"Vaishnavi", age:19, marks:92, city:"Kanpur"}
{roll:8, name:"Pratik", age:19, marks:90, city:"Lucknow"}
{roll:12, name:"Shreya", age:19, marks:95, city:"Lucknow"}
{roll:14, name:"Shruti", age:19, marks:87, city:"Prayagraj"}
{roll:16, name:"Sejal", age:18, marks:89, city:"Lucknow"}
{roll:19, name:"Harshita", age:20, marks:91, city:"Lucknow"}
{roll:22, name:"Saumya", age:19, marks:86, city:"Jaunpur"}
{roll:26, name:"Anushka", age:20, marks:94, city:"Lucknow"}
{roll:27, name:"Ananya", age:19, marks:88, city:"Varanasi"}
{roll:32, name:"Anika", age:19, marks:90, city:"Lucknow"}
{roll:35, name:"Krishna", age:19, marks:96, city:"Prayagraj"}
{roll:39, name:"Dhruv", age:20, marks:93, city:"Lucknow"}
{roll:42, name:"Akanksha", age:20, marks:89, city:"Lucknow"}
{roll:48, name:"Priya", age:20, marks:87, city:"Lucknow"}
{roll:50, name:"Neha", age:18, marks:91, city:"Varanasi"}
```

Explanation: Finds students whose marks are greater than 85.

11. LESS THAN — \$lt

MongoDB query shown as a terminal-style screenshot:



```
MongoDB Shell | $lt - Less than

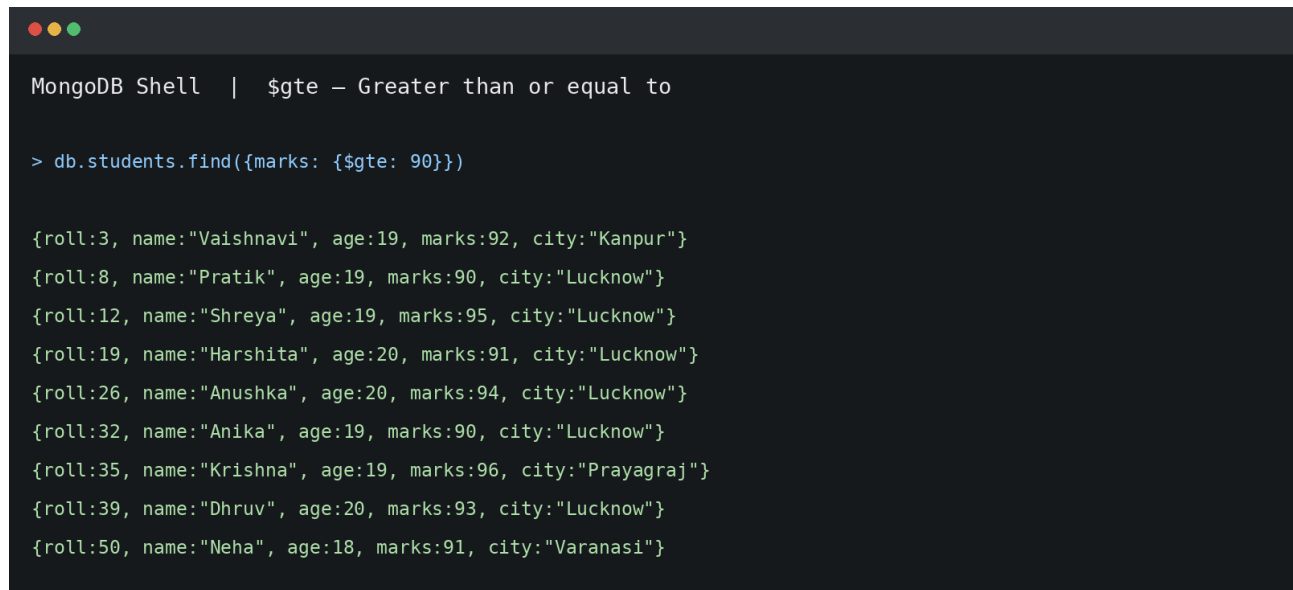
> db.students.find({marks: {$lt: 60}})

{roll:18, name:"Jai", age:18, marks:58, city:"Gonda"}
{roll:33, name:"Jai", age:20, marks:55, city:"Ayodhya"}
{roll:47, name:"Aman", age:18, marks:59, city:"Gonda"}
```

Explanation: Finds students whose marks are less than 60.

12. GREATER THAN OR EQUAL TO — \$gte

MongoDB query shown as a terminal-style screenshot:



```
MongoDB Shell | $gte – Greater than or equal to

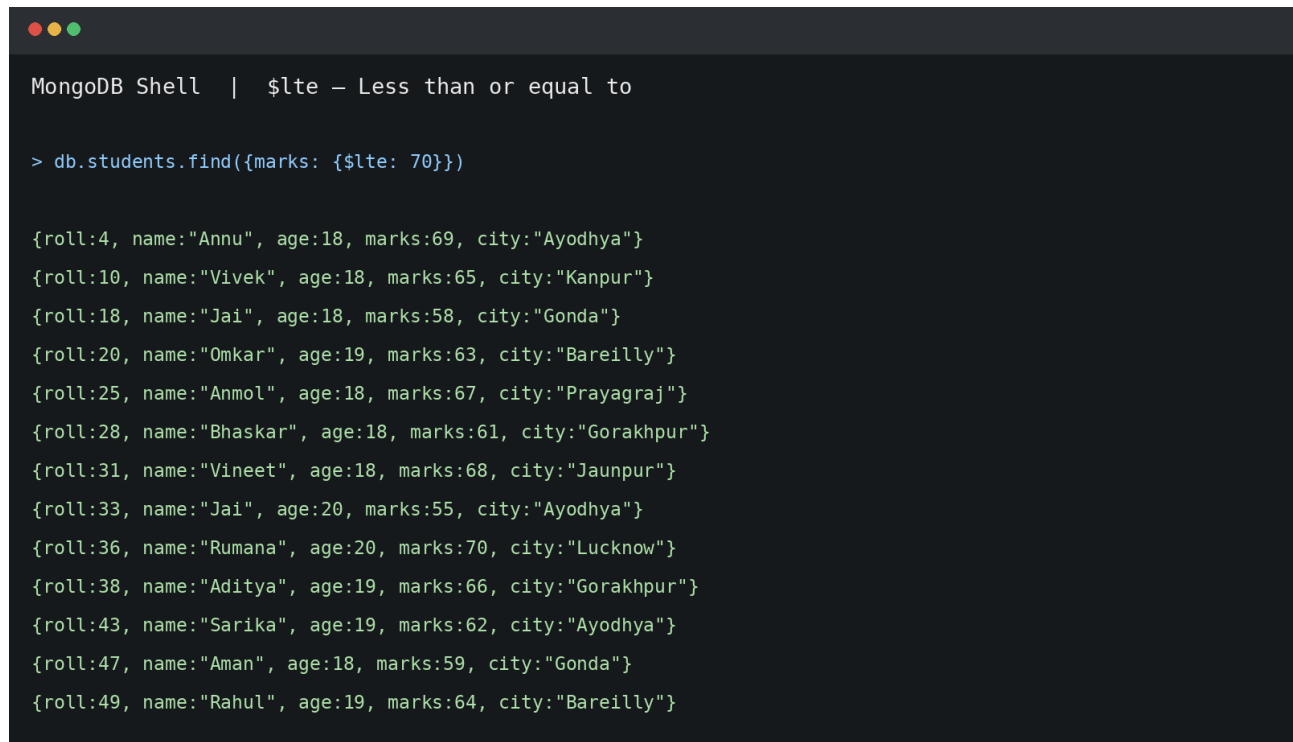
> db.students.find({marks: {$gte: 90}})

{roll:3, name:"Vaishnavi", age:19, marks:92, city:"Kanpur"}
{roll:8, name:"Pratik", age:19, marks:90, city:"Lucknow"}
{roll:12, name:"Shreya", age:19, marks:95, city:"Lucknow"}
{roll:19, name:"Harshita", age:20, marks:91, city:"Lucknow"}
{roll:26, name:"Anushka", age:20, marks:94, city:"Lucknow"}
{roll:32, name:"Anika", age:19, marks:90, city:"Lucknow"}
{roll:35, name:"Krishna", age:19, marks:96, city:"Prayagraj"}
{roll:39, name:"Dhruv", age:20, marks:93, city:"Lucknow"}
{roll:50, name:"Neha", age:18, marks:91, city:"Varanasi"}
```

Explanation: Finds students whose marks are 90 or above.

13. LESS THAN OR EQUAL TO — \$lte

MongoDB query shown as a terminal-style screenshot:



```
MongoDB Shell | $lte – Less than or equal to

> db.students.find({marks: {$lte: 70}})

{roll:4, name:"Annu", age:18, marks:69, city:"Ayodhya"}
{roll:10, name:"Vivek", age:18, marks:65, city:"Kanpur"}
{roll:18, name:"Jai", age:18, marks:58, city:"Gonda"}
{roll:20, name:"Omkar", age:19, marks:63, city:"Bareilly"}
{roll:25, name:"Anmol", age:18, marks:67, city:"Prayagraj"}
{roll:28, name:"Bhaskar", age:18, marks:61, city:"Gorakhpur"}
{roll:31, name:"Vineet", age:18, marks:68, city:"Jaunpur"}
{roll:33, name:"Jai", age:20, marks:55, city:"Ayodhya"}
{roll:36, name:"Rumana", age:20, marks:70, city:"Lucknow"}
{roll:38, name:"Aditya", age:19, marks:66, city:"Gorakhpur"}
{roll:43, name:"Sarika", age:19, marks:62, city:"Ayodhya"}
{roll:47, name:"Aman", age:18, marks:59, city:"Gonda"}
{roll:49, name:"Rahul", age:19, marks:64, city:"Bareilly"}
```

Explanation: Finds students whose marks are 70 or below.

14. UPDATE ONE OPERATION

MongoDB query shown as a terminal-style screenshot:

A terminal window with a dark background and light-colored text. The title bar at the top shows three colored circles (red, yellow, green) and the text "MongoDB Shell | Update one". The terminal content shows a MongoDB query being executed, followed by a JSON response indicating the update was successful. The response includes fields for acknowledged, matchedCount, and modifiedCount, and a line showing the updated document with its fields: roll, name, age, marks, and city.

```
MongoDB Shell | Update one

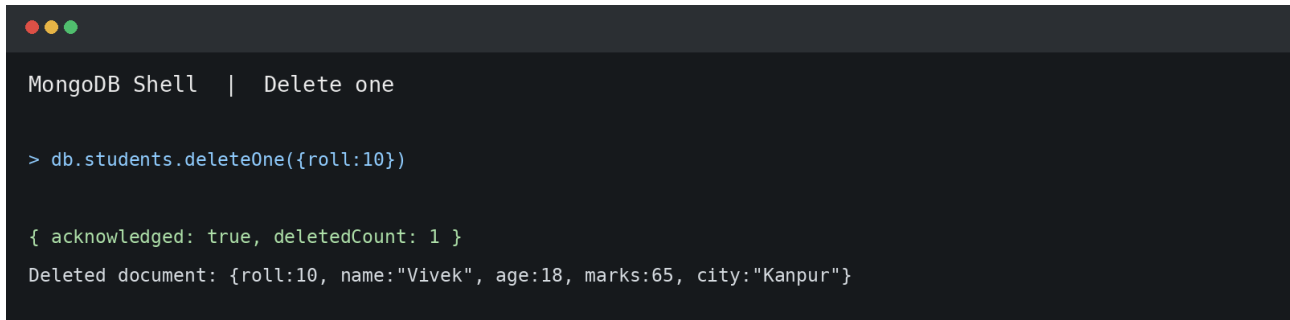
> db.students.updateOne({roll:5}, {$set:{marks:95}})

{ acknowledged: true, matchedCount: 1, modifiedCount: 1 }
Updated document: {roll:5, name:"Vansh", age:19, marks:95, city:"Varanasi"}
```

Explanation: Updates the marks to 95 for the student with roll number 5.

15. DELETE ONE OPERATION

MongoDB query shown as a terminal-style screenshot:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The title bar reads "MongoDB Shell | Delete one". The terminal shows a command being executed, its output, and a confirmation message.

```
MongoDB Shell | Delete one

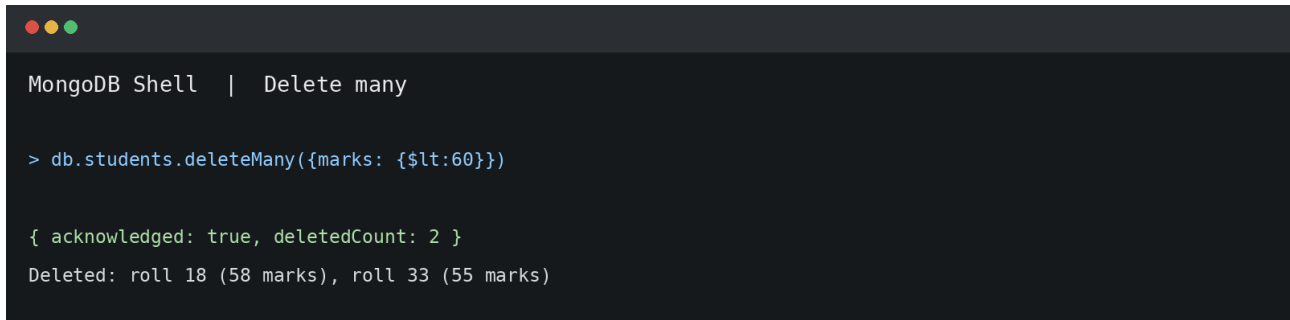
> db.students.deleteOne({roll:10})

{ acknowledged: true, deletedCount: 1 }
Deleted document: {roll:10, name:"Vivek", age:18, marks:65, city:"Kanpur"}
```

Explanation: Deletes one document matching roll number 10.

16. DELETE MANY OPERATION

MongoDB query shown as a terminal-style screenshot:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The title bar reads "MongoDB Shell | Delete many". The terminal shows a MongoDB command being executed, followed by a JSON response and a human-readable summary of the deletion.

```
MongoDB Shell | Delete many

> db.students.deleteMany({marks: {$lt:60}})

{ acknowledged: true, deletedCount: 2 }
Deleted: roll 18 (58 marks), roll 33 (55 marks)
```

Explanation: Deletes all students having marks below 60.

17. AND OPERATION — \$and

MongoDB query shown as a terminal-style screenshot:

```
MongoDB Shell | AND – Both conditions

> db.students.find({$and:[{marks:{$gt:80}}, {age:19}]})

{roll:1, name:"Anamika", age:19, marks:88, city:"Lucknow"}
{roll:3, name:"Vaishnavi", age:19, marks:92, city:"Kanpur"}
{roll:5, name:"Vansh", age:19, marks:81, city:"Varanasi"}
{roll:8, name:"Pratik", age:19, marks:90, city:"Lucknow"}
{roll:12, name:"Shreya", age:19, marks:95, city:"Lucknow"}
{roll:14, name:"Shruti", age:19, marks:87, city:"Prayagraj"}
{roll:17, name:"Astha", age:19, marks:82, city:"Kanpur"}
{roll:22, name:"Saumya", age:19, marks:86, city:"Jaunpur"}
{roll:27, name:"Ananya", age:19, marks:88, city:"Varanasi"}
{roll:32, name:"Anika", age:19, marks:90, city:"Lucknow"}
{roll:35, name:"Krishna", age:19, marks:96, city:"Prayagraj"}
{roll:40, name:"Nivedita", age:19, marks:85, city:"Kanpur"}
```

Explanation: Finds students satisfying both conditions.

18. OR OPERATION — \$or

MongoDB query shown as a terminal-style screenshot:

```
MongoDB Shell | OR – Either condition

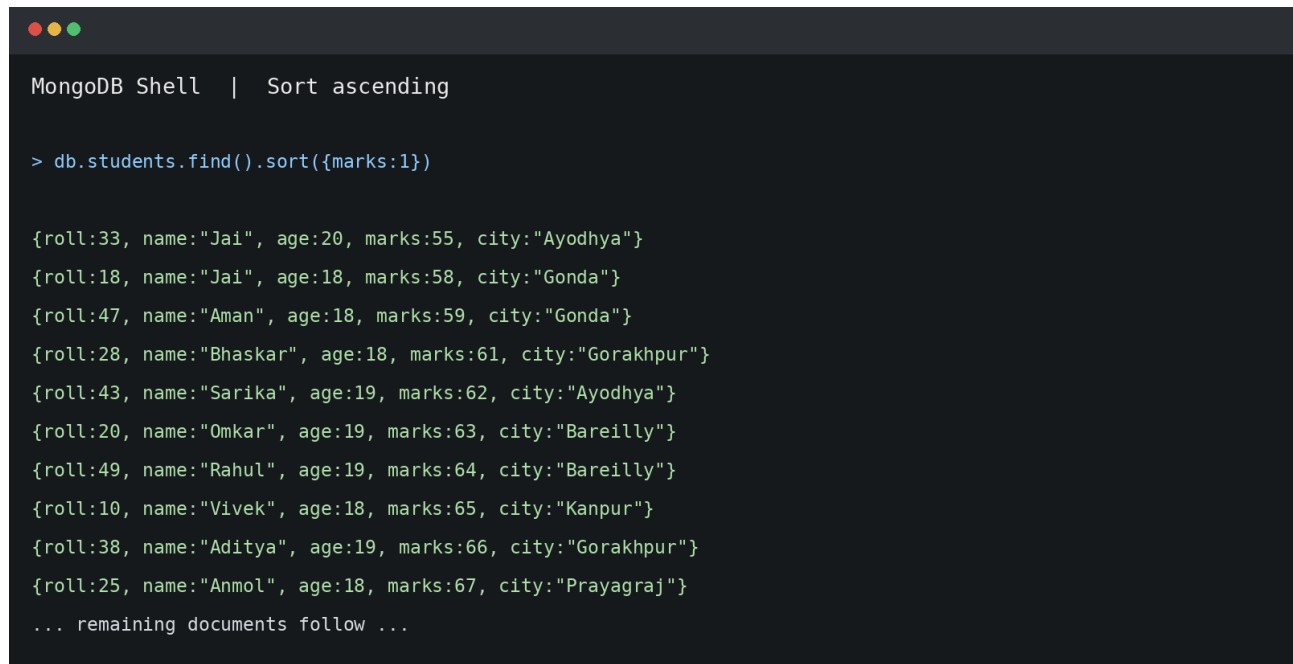
> db.students.find({$or:[{city:"Lucknow"}, {city:"Kanpur"}]})

{roll:1, name:"Anamika", age:19, marks:88, city:"Lucknow"}
{roll:3, name:"Vaishnavi", age:19, marks:92, city:"Kanpur"}
{roll:6, name:"Amrita", age:20, marks:85, city:"Lucknow"}
{roll:8, name:"Pratik", age:19, marks:90, city:"Lucknow"}
{roll:10, name:"Vivek", age:18, marks:65, city:"Kanpur"}
{roll:12, name:"Shreya", age:19, marks:95, city:"Lucknow"}
{roll:16, name:"Sejal", age:18, marks:89, city:"Lucknow"}
{roll:17, name:"Astha", age:19, marks:82, city:"Kanpur"}
{roll:19, name:"Harshita", age:20, marks:91, city:"Lucknow"}
{roll:21, name:"Ayush", age:18, marks:77, city:"Lucknow"}
... more matching documents ...
```

Explanation: Finds students from Lucknow or Kanpur.

19. SORT OPERATION — ASCENDING

MongoDB query shown as a terminal-style screenshot:

A terminal window with a dark background and light-colored text. The title bar at the top shows three colored dots (red, yellow, green) on the left. The terminal text shows a MongoDB Shell prompt with a command to sort a collection of students by marks in ascending order. The output lists ten student documents, sorted by their marks from 55 to 67. The documents include fields for roll number, name, age, marks, and city. The last line of the output indicates that more documents follow.

```
MongoDB Shell | Sort ascending

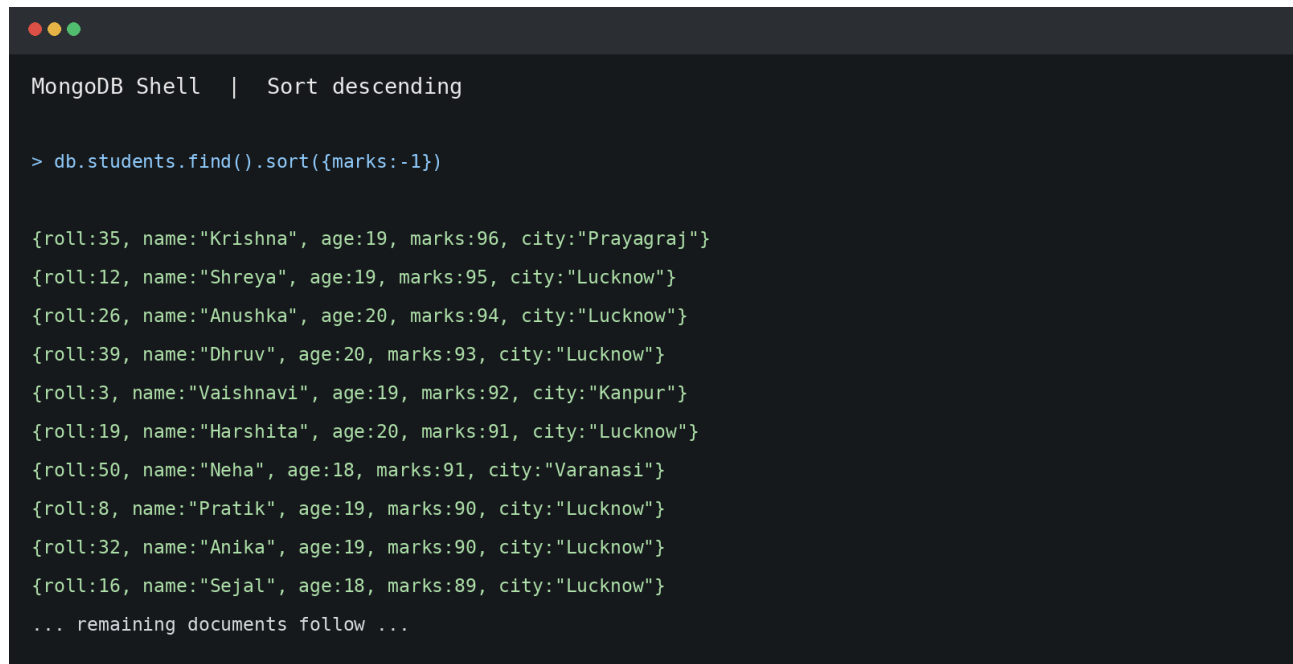
> db.students.find().sort({marks:1})

{roll:33, name:"Jai", age:20, marks:55, city:"Ayodhya"}
{roll:18, name:"Jai", age:18, marks:58, city:"Gonda"}
{roll:47, name:"Aman", age:18, marks:59, city:"Gonda"}
{roll:28, name:"Bhaskar", age:18, marks:61, city:"Gorakhpur"}
{roll:43, name:"Sarika", age:19, marks:62, city:"Ayodhya"}
{roll:20, name:"Omkar", age:19, marks:63, city:"Bareilly"}
{roll:49, name:"Rahul", age:19, marks:64, city:"Bareilly"}
{roll:10, name:"Vivek", age:18, marks:65, city:"Kanpur"}
{roll:38, name:"Aditya", age:19, marks:66, city:"Gorakhpur"}
{roll:25, name:"Anmol", age:18, marks:67, city:"Prayagraj"}
... remaining documents follow ...
```

Explanation: Sorts students by marks from lowest to highest. Use 1 for ascending order.

20. SORT OPERATION — DESCENDING

MongoDB query shown as a terminal-style screenshot:



```
MongoDB Shell | Sort descending

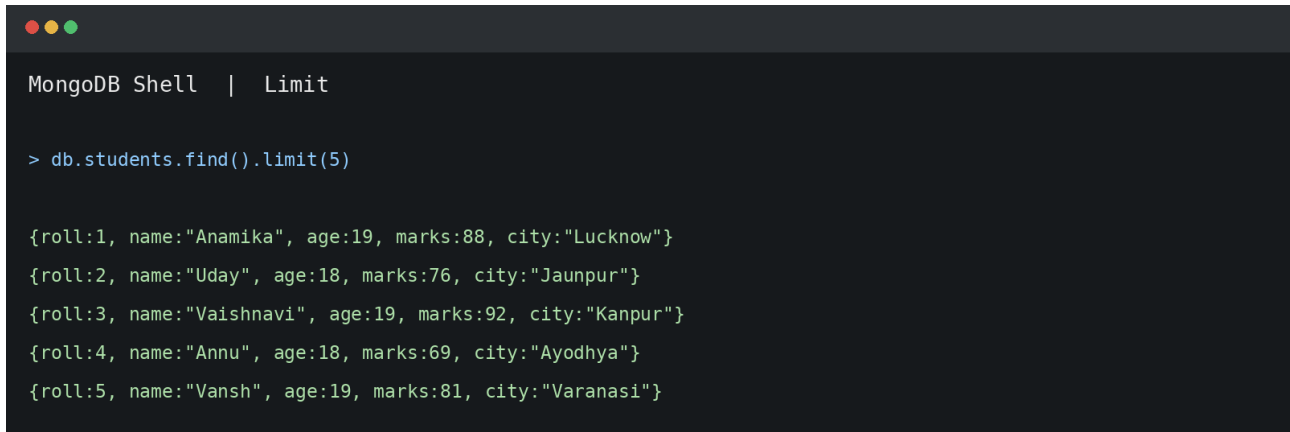
> db.students.find().sort({marks:-1})

{roll:35, name:"Krishna", age:19, marks:96, city:"Prayagraj"}
{roll:12, name:"Shreya", age:19, marks:95, city:"Lucknow"}
{roll:26, name:"Anushka", age:20, marks:94, city:"Lucknow"}
{roll:39, name:"Dhruv", age:20, marks:93, city:"Lucknow"}
{roll:3, name:"Vaishnavi", age:19, marks:92, city:"Kanpur"}
{roll:19, name:"Harshita", age:20, marks:91, city:"Lucknow"}
{roll:50, name:"Neha", age:18, marks:91, city:"Varanasi"}
{roll:8, name:"Pratik", age:19, marks:90, city:"Lucknow"}
{roll:32, name:"Anika", age:19, marks:90, city:"Lucknow"}
{roll:16, name:"Sejal", age:18, marks:89, city:"Lucknow"}
... remaining documents follow ...
```

Explanation: Sorts students by marks from highest to lowest. Use -1 for descending order.

21. LIMIT OPERATION

MongoDB query shown as a terminal-style screenshot:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The title bar reads "MongoDB Shell | Limit". The prompt is ">". The command entered is "db.students.find().limit(5)". The output consists of five JSON documents, each on a new line, representing the first five records in the 'students' collection.

```
MongoDB Shell | Limit

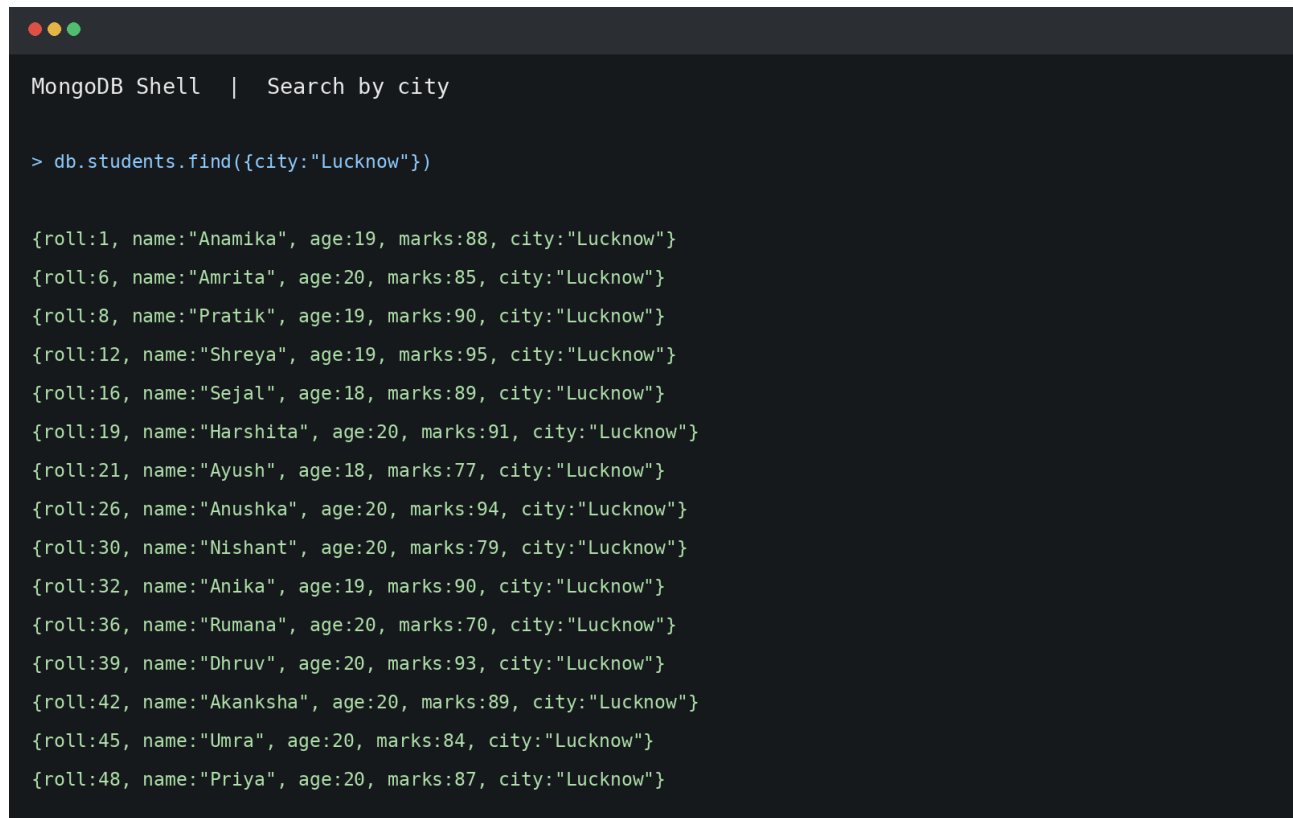
> db.students.find().limit(5)

{roll:1, name:"Anamika", age:19, marks:88, city:"Lucknow"}
{roll:2, name:"Uday", age:18, marks:76, city:"Jaunpur"}
{roll:3, name:"Vaishnavi", age:19, marks:92, city:"Kanpur"}
{roll:4, name:"Annu", age:18, marks:69, city:"Ayodhya"}
{roll:5, name:"Vansh", age:19, marks:81, city:"Varanasi"}
```

Explanation: Displays only the first five documents.

22. SEARCH BY CITY

MongoDB query shown as a terminal-style screenshot:

A terminal window titled "MongoDB Shell | Search by city" displays the command > db.students.find({city:"Lucknow"}) and its output. The output consists of 15 JSON documents, each representing a student with fields for roll number, name, age, marks, and city. The students listed are Anamika, Amrita, Pratik, Shreya, Sejal, Harshita, Ayush, Anushka, Nishant, Anika, Rumana, Dhruv, Akanksha, Umra, and Priya, all from Lucknow.

```
MongoDB Shell | Search by city

> db.students.find({city:"Lucknow"})

{roll:1, name:"Anamika", age:19, marks:88, city:"Lucknow"}
{roll:6, name:"Amrita", age:20, marks:85, city:"Lucknow"}
{roll:8, name:"Pratik", age:19, marks:90, city:"Lucknow"}
{roll:12, name:"Shreya", age:19, marks:95, city:"Lucknow"}
{roll:16, name:"Sejal", age:18, marks:89, city:"Lucknow"}
{roll:19, name:"Harshita", age:20, marks:91, city:"Lucknow"}
{roll:21, name:"Ayush", age:18, marks:77, city:"Lucknow"}
{roll:26, name:"Anushka", age:20, marks:94, city:"Lucknow"}
{roll:30, name:"Nishant", age:20, marks:79, city:"Lucknow"}
{roll:32, name:"Anika", age:19, marks:90, city:"Lucknow"}
{roll:36, name:"Rumana", age:20, marks:70, city:"Lucknow"}
{roll:39, name:"Dhruv", age:20, marks:93, city:"Lucknow"}
{roll:42, name:"Akanksha", age:20, marks:89, city:"Lucknow"}
{roll:45, name:"Umra", age:20, marks:84, city:"Lucknow"}
{roll:48, name:"Priya", age:20, marks:87, city:"Lucknow"}
```

Explanation: Finds all students whose city is Lucknow.

23. COUNT DOCUMENTS

MongoDB query shown as a terminal-style screenshot:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The title bar reads "MongoDB Shell | Count documents". The prompt ">" is followed by the command "db.students.countDocuments()". The output "48" is displayed on the next line.

```
MongoDB Shell | Count documents

> db.students.countDocuments()

48
```

Explanation: Counts documents currently present after the delete operations.

24. MONGODB OPERATORS USED

Operator	Meaning	Example
\$eq	Equal to	marks: {\$eq:88}
\$gt	Greater than	marks: {\$gt:85}
\$lt	Less than	marks: {\$lt:60}
\$gte	Greater than or equal to	marks: {\$gte:90}
\$lte	Less than or equal to	marks: {\$lte:70}
\$and	All conditions true	\$and:[...]
\$or	At least one condition true	\$or:[...]

25. CONCLUSION

This project provided practical knowledge of NoSQL databases using MongoDB. A Student Management System containing student information was created and different MongoDB operations were performed on the data.

Through this project, I learned how to insert, retrieve, update and delete documents. I also learned how ascending and descending sorting works, how comparison operators such as \$eq, \$gt, \$lt, \$gte and \$lte work, and how logical operators such as \$and and \$or can be used to filter data.

Overall, this project helped me understand how MongoDB can be used to manage and query student data efficiently.

26. REFERENCES

1. MongoDB documentation and MongoDB Shell concepts.
2. Babu Banarasi Das University NoSQL practical/reference document supplied for this project.
3. Course notes and classroom guidance.

APPENDIX — PROJECT DETAILS

Project: Student Management System Using MongoDB

Database: Students

Collection: students

Initial records: 50

Student: Diwakar Mishra

Roll No.: 33 (Enrollment No. 1250258198)

Class: BCA DS & AI 23

Teacher: Mr. Harendra Singh Rajpoot

Note: The query panels in this report are formatted as screenshot-style representations so that the report presents the practical work visually rather than as raw code blocks. The project content and dataset are based on the supplied reference document.